Clase 17 · Boosting y Gradient Boosting

Reconocimiento de Patrones · Ing. Rubén Omar Azuara Domínguez · viernes 23 oct 2026 (sesión 20 del curso) · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Repaso: CART, Random Forest y Bagging

Un árbol **CART** (Classification and Regression Trees) divide el espacio de características con reglas binarias (nodo izquierdo/derecho) minimizando una impureza (Gini o entropía).

- **Raíz:** la división que más información aporta al inicio. **Nodos internos:** evalúan umbrales de una característica. **Hojas:** predicción según la proporción de clases. **Ramas:** caminos de la raíz a las hojas.
- Hiperparámetros: profundidad máxima (`max_depth`), mínimo de muestras para dividir y número de características exploradas.

Random Forest es un ensamble de árboles que vota por mayoría. Para que los árboles sean distintos se usa **Bagging** (Bootstrap Aggregating): muestrear N datos al azar, uniforme y **con reemplazo**.

> **Muestras Out-of-Bag**
>
> Probabilidad de no elegir una muestra en N extracciones: $(1 - 1/N)^N \to e^{-1} \approx 0.368$. Cada árbol ve ≈ 63.2 % de los datos; el 36.8 % restante (**OOB**) sirve para evaluarlo.

Bagging reduce la varianza. Error común: no muestrear con reemplazo o no limitar las características por nodo, y todos los árboles salen iguales.

2. Boosting: ensamble secuencial

Los modelos base (aprendices débiles, árboles poco profundos) se entrenan **uno tras otro** y cada uno corrige los errores de los anteriores.

1. Entrenar $h_0(x)$ con los datos originales.
2. Identificar las muestras mal clasificadas y calcular el residuo.
3. Dar más peso a los ejemplos donde se falló.
4. Entrenar $h_1(x)$ para predecir esos errores.
5. Repetir M veces (convergencia o máximo de árboles).
6. Predicción final = suma ponderada de las predicciones secuenciales.

Pregunta en vivo: ¿quién identifica los errores? Nadie a mano: el código calcula la diferencia entre etiqueta y predicción (y = 1, predijo 0.4 → error 0.6) y ese residuo es el objetivo del siguiente modelo.

Ensamble	Cómo combina
Bagging (Random Forest)	Modelos en **paralelo** con muestras bootstrap; voto o promedio; reduce varianza

Boosting	Modelos en **secuencia**, cada uno sobre los residuos del anterior
Stacking	Un metamodelo combina predicciones en dos niveles

3. Gradient Boosting

> **Idea central**
> Boosting como **descenso de gradiente en el espacio de funciones**: no se ajustan pesos, se agregan árboles que predicen el **gradiente negativo** de la pérdida.

- Expansión aditiva: $f_M(x) = \Sigma_{m=0..M} \beta_m h_m(x)$; en cada paso $f_m(x) = f_{m-1}(x) + \beta_m h_m(x)$, buscando $\beta_m h_m(x_i) = y_i - f_{m-1}(x_i)$.
- Con pérdida cuadrática $L = \frac{1}{2}(y - f(x))^2$: $\mathbf{-\partial L/\partial f(x_i) = y_i - f(x_i)}$ → el residuo es exactamente el negativo del gradiente.
- Jerome Friedman: para cualquier pérdida diferenciable, ese gradiente negativo es un **pseudo-residuo** r_{im} al que se ajusta un árbol por mínimos cuadrados.
- **Tasa de aprendizaje** ν ($0 < \nu \leq 1$) contra el sobreajuste: $f_m(x) = f_{m-1}(x) + \nu \cdot \Sigma_j \gamma_{jm} \cdot I(x \in R_{jm})$, con γ_{jm} el valor óptimo de la hoja j y R_{jm} su región.

Contraste: en regresión logística y redes neuronales el gradiente actualiza **pesos**; aquí actualiza la **función**.

4. Ejercicio resuelto: ¿compra según la edad? (binario)

Paso	Cálculo
Inicialización	$p_0 = 0.5$ para todos → log-odds $f_0 = 0$
Residuos iniciales	compró: $1 - 0.5 = +0.5$; no compró: $0 - 0.5 = -0.5$
Árbol 1	Regla Edad < 32. Hoja izquierda (ejemplos 1, 4, 6): residuos −0.5; derecha: +0.5
Valor de hoja	$\gamma = \Sigma$ residuos / $\Sigma\, p(1-p)$. Izq.: $-1.5 / 0.75 =$ **−2**; der.: $+1.5 / 0.75 =$ **+2**
Actualización ($\nu = 0.5$)	izq.: $f_1 = 0 + 0.5(-2) = -1$; der.: $f_1 = 0 + 0.5(2) = +1$
Sigmoide $\sigma(z) = 1/(1 + e^{-z})$	$\sigma(-1) \approx$ **0.27**; $\sigma(+1) \approx$ **0.73**
Nuevos residuos	$y = 0$: −0.27; $y = 1$: +0.27 (bajaron de ±0.5: converge)

Error común: olvidar que γ está en log-odds y hay que pasarlo por la sigmoide para tener probabilidad.

5. Multiclase y softmax

Con $K > 2$ clases se construyen **K secuencias de árboles** (una por clase, uno contra el resto) sobre la variable en **one-hot** (columnas Gato, Perro, Caballo).

- Inicialización con 6 ejemplos (2 por clase): $f_{0,k} = \ln(2/6) = -1.099$ → probabilidad $1/3 \approx 0.333$ cada una.

- Residuos iniciales: clase correcta 1 − 0.333 = +0.667; las otras 0 − 0.333 = −0.333. En cada paso se entrenan 3 árboles.

Softmax

P(Y = k | x) = e^{f_k(x)} / Σ_j e^{f_j(x)}. Con log-odds 2, 1 y 0.5: $e^2 \approx 7.389$, $e^1 \approx 2.718$, e^0.5 ≈ 1.6487; suma ≈ 11.756 → P ≈ **0.63, 0.23 y 0.14** (suman 1).

Es la misma capa de salida que usan las redes neuronales multiclase.

6. Ventajas, limitaciones y XGBoost

- **Ventajas:** excelente en datos tabulares (domina Kaggle; se usa en Hugging Face) e interpretable (árboles e importancia de características).
- **Secuencial:** cada árbol espera al anterior; no escala bien sin optimizaciones.
- **Sin estructura espacial ni temporal:** no capta vecindad de píxeles ni dependencia temporal de audio o series.
- **Datos:** necesita volúmenes grandes para converger, a diferencia de Naive Bayes o SVM.
- **XGBoost** (Extreme Gradient Boosting): cómputo eficiente, poda inteligente y regularización L1 y L2 en la pérdida.

Esto justifica pasar a redes convolucionales (imágenes) y Transformers (audio, lenguaje).

7. Avisos

- **Práctica 4 (KNN y regresión logística):** KNN con fit/predict y distancia de Minkowski (parámetro p), probando varios k, datos con y sin normalizar; regresión logística con fit/predict/predict_proba sobre Breast Cancer, L1 y L2, matriz de confusión, métricas y pesos θ. Fechas sin contrastar con Classroom.
- **Proyecto final:** emociones en imágenes, emociones en audio o comandos de voz (detalle en la Clase 18).
- Sigue el último bloque: redes neuronales y luego Transformers.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Bagging vs. Boosting. Bagging: árboles en paralelo con bootstrap (con reemplazo), voto/promedio, baja la varianza; ≈ 36.8 % de datos OOB por árbol. Boosting: árboles en secuencia sobre los residuos del anterior.

> **Gradient Boosting (examen)**
>
> Descenso de gradiente en el **espacio de funciones** (optimiza f(x), no pesos). Con pérdida cuadrática, residuo = −gradiente: $y_i - f(x_i)$. Actualización: f_m = f_{m−1} + ν·γ.

Fórmula	Para recordar
Valor de hoja	γ = Σ residuos / Σ p(1 − p)
Ejemplo	3 residuos de −0.5 con p = 0.5 → −1.5 / 0.75 = −2
Log-odds → probabilidad	$\sigma(z) = 1/(1 + e^{-z})$: σ(±1) ≈ 0.73 / 0.27
Multiclase	K secuencias + softmax; (2, 1, 0.5) → 0.63, 0.23, 0.14

Limitaciones. Lento por ser secuencial, pide muchos datos y no capta estructura espacial (imágenes) ni temporal (audio). XGBoost agrega eficiencia, poda y regularización L1/L2.